Anti-User Technology: The Software that Hinders Software

Software is a core component of the modern era; it is the foundation of what today's society sits and relies upon. Our daily lives and routines are both defined and outlined by technological advancements. Software finds itself intertwined deeply within every crevice of society. One would have to actively attempt to separate themselves from software if they were to try to avoid it; It is almost inescapable in this day and age. It empowers industries such as film, music, agriculture, appliances, travel, communication, and many others. Such industries owe their prosperity to software, as traditional tools are outclassed in almost every way. Even human connection can attribute its strength to software; We utilize social networking software to form bonds across continents, peer into the lives of others, and to speak our minds within fast-paced blips of content.

Yet at the end of the day, the miracles that software performs in our lives are just a set of instructions for computers to compute. Behind the miracle of computing, there is an unforgiving sheet of binary, or ordered zeros and ones, that controls some hardware's behavior. A software's binary comes from a compiler, which accepts human-readable code as input and outputs machine usable binary. Even though this binary I speak of is completely intangible, machines use it to create tangible differences to our society. With little data, software creates huge ripple effects between country and country, person and person.

Such influence and possibility are what has driven me to pursue this subject. My interest has since expanded when I was promised of the vast horizon that software offers to humanity as a whole. Many incumbent leaders of technology preach of seemingly impossible promises. Such that all of humanity's unsolvable problems can seek solution software. More specifically, the software they sell.

Many of today's software leaders, the people who provide the vast majority of programs we use on the clock, are entrepreneurs. Entrepreneurs, more often than not, seek out one thing: profit. It should come to nobody's surprise that a significant sum of products are pushed more for "helping their bottom line" than the idealism of "helping the world", as harsh as that sounds. Granted, it is expensive to produce software, but there comes a point between achieving promise and profit. This unavoidable goal of software to make profit has intrigued me: How has this underlying goal disrupted software's connection in society? Has it hindered technology's ability to carry out its promises? Overall, I seek to answer: *How has for-profit software undermined technology's potential?*

Though software has a relatively shallow history, its impact blows many other innovations out of proportion. It can be dated back to the 19th century with Ada Lovelace's programs for Charles Babbage's analytical engine. However, digital software was revolutionized in the late 1940s, being written directly in binary code. Binary code is a set of instructions made of zeros and ones to instruct a computer to perform a various set of tasks. Initially, software was utilized for mathematical calculations, processing numbers, and automating simple monotonous tasks. As it progressed further into the modern era however, software became interwoven with everyday tasks and challenges, outside of mathematical operation. Many industries, lives, and standards were being set by software's influence. Despite early software taking upwards of days to compute outputs, time developed impactful software even in its beginning: "technologies have allowed people to shape the physical world and the world of knowledge to meet their needs and wants, to extend the reach of their bodies, hands, and minds, to span rivers, and to traverse continents" ("Technology and Society"). Software has been able to perform extensive miracles. Long distance communication became plausible with the development of interconnectivity

between computers. Maturing methods of transportation were managed by software systems, including planes, cars and even rocket ships. The 1969 moon landing was reliant on software, establishing software's application in transportation. Even jobs, though some were taken, many were bloomed through software's potential. Alongside software's growth came many monetary opportunities. As software became more accessible with the home computer's widespread use, many people found potential in economic gain. In today's era, said economic gain is expressed significantly, juxtaposing software's first steps of commercialization. In my interview with Mr. Ekberg, a retired software engineer with 28 years of experience, he expressed his perspective with software's commercialization:

> Lately, things really changed and a lot of software started to cost money. Having to pay for things like: Tools that were used during the build process, bug tracking systems, project management systems, infrastructure operating system support, editing tools that we would use, in order to access certain features. Suddenly it takes money.

In the earliest days of software's commercialization, the viability to distribute software was relatively poor. Software was sold through disks, microtransactions had yet to exist, and computer hardware was very limited. In recent years however, software has changed drastically. Modern software comes pre-packaged with devices. Programs can be sold and downloaded over the internet. Furthermore, some programs have monthly subscriptions, licensing, fees and more bundled alongside its functionality. Leaders in the software industry, unsurprisingly, are valued over trillions of dollars and for good reason; The programs we rely on in our devices are ones distributed by these powerful few. These monetary barriers in the software industry however highlight the drive for profit. This invokes the question: How has the drive for monetary gain burdened software's potential?

For profit software has actively hindered the ability for technology to prosper. Software has only been able to progress to today's uses and applications because of innovation. Yet one cannot innovate with software if they simply do not have access to the modern tooling and knowledge needed to do so. The latest set of professional grade tools, software, and innovations are often difficult to afford. As a result, individuals, small teams and even some businesses are limited in their ability to contribute to modern technology: "The companies will basically charge whatever they think they can get away with… It wasn't just 25 dollars per seat per developer for access to some product, it might be fifteen hundred dollars for a license, per developer per month in some cases" (Ekberg). Modern tooling, even within industries unrelated to software development, can be very expensive. With monthly subscriptions and predatory licenses, this holds especially true. Furthermore, proprietary software is closed source, meaning that one cannot build off of, edit or modify the program they purchased. These barriers fundamentally burden the ability to innovate and revolutionize technology in the modern era. A recent example involves Google restricting what users are permitted to run and install on their devices: "[Apps] will need to come from a developer who has gone through Google's new verification process… [which] requires sharing personal identifying information like legal name, address, and phone number" (Rudra). Innovation prospers in environments that support collaborative development, freedom over devices and accessibility to knowledge and tools. By limiting the access to freedom, tools and knowledge only to a small pool of entities, or those who can afford access, innovation is thus limited.

Alongside burdening innovation, flexibility and freedom is hindered by the nature of proprietary software: "With proprietary software, the program controls the users, and some other entity controls the program. So the proprietary program gives its developer power over its users.

That is unjust in itself; moreover, it tempts the developer to mistreat the users in other ways" (Stallman). A common form of this mistreatment vendor lock-in. Vendor lock-in enforces users to strictly use a particular vendor's goods or services. For example, by making a program only accessible to one manufacturer's products, all users of said program can only use said products. This is possible through the closed source nature of proprietary software, preventing others from ensuring compatibility, flexibility and accessibility. Privacy is also compromised in the usage of for-profit software solutions. Without the ability to see the source code of a program, there is no transparency established between the program and the user. In a sense, the user must trust that the program does only what it states it will do. In many cases, embedded functionality is utilized to exploit user data that is personally identifiable. This data is then sold to data brokers for a large sum of money, to then utilize such data to form educated attacks on specific people. A prominent example of this resides in tech education, where a child's mind is essentially sold by tech corporations. In his article, "Technology Might be Making Education Worse," Antero Garcia argues that educational software of which exploits data, harms more than just the privacy of children: "Given the substantial amount of scholarly time I've invested in documenting the life-changing possibilities of digital technologies, it gives me no pleasure to suggest that these tools might be slowly poisoning us" (Garcia). Data exploitation extends past the confines of educational tech. It is similarly embedded in social media applications for the same purpose. Proprietary software solutions collect user data, either to sell or to use against the user. It becomes an implicit reality to people that to participate in a civic community, one must sacrifice their privacy. Handing over personal information to software leverages an entity's ability to sow bias, employ disinformation, and to target people with private data. Despite the overbearing amount of issues, inescapable monetary barriers and privacy destroying programs, there is a

potential solution. A solution of which can eliminate the grievances of proprietary software's nature.

Contrasting proprietary software, open source software (OSS) directly hits what for-profit fundamentally misses. It can be essentially viewed as software without the grievances of proprietary programs. Even though proprietary software dominates the technology we rely on in our daily lives, OSS is becoming more essential and viable every day. The OSS movement started as a counter to proprietary computing. Programs that are open-sourced are pieces of software whose source code is completely available to the public. This aspect enables communities to edit, modify and utilize OSS to their own needs and requirements. An instance of OSS's power is highlighted by Scott Giddings, the founder of the Bay Area I.T. business Cyberdoc. Giddings is an internet technician, having been in the industry for thirty years. He believes that OSS is a powerful source of technological progression: "It's a wonderful place for people to learn, grow, share experiences, and improve things. From that perspective, I think it's terrific." As a result of for-profit's closed source nature, it becomes difficult for people to build off of modern software. Monopolistic corporations remain competitive through hiding and patenting their solutions, stifling others from snowballing their inventions. Beyond empowering learning, software growth and community, OSS can serve as a more efficient and secure alternative to proprietary tech solutions. Too often is openness seen as a vulnerability. Having many eyes on the inner-workings of a program might seem dangerous, but that is a misconception. Transparency is what prevents programs from being malicious, in contrast to closed-source software. Alongside this benefit, "it's widely used. There's a large number of people exercising that code and testing it out. It's more likely that problems will be discovered and fixed" (Ekberg). Having a significant portion of people using and reviewing software holds

everyone accountable to everyone else. If someone finds a problem with the program, they can make edits to amend it. Even if they are not a programmer, they can raise awareness of the issue to developers to get it fixed for everyone. Since the software's functionality is additionally open to all, everyone can be assured as to what the program does. Such transparency directly prevents others from inserting potentially dangerous code to harm others. This vast array of benefits are the idealisms of which proprietary software cannot obtain. The idealistic nature of OSS however, tends to fall short in a variety of cases; proprietary software retains benefits OSS struggles to hold. Looking past its shortcomings, some principals and ideals of proprietary software are justifiable. Aspects of for-profit software achieve progression in technological innovation and accessibility. Programs developed by tech firms, though their goal tends to be to increase shareholder profits, are made to reach mass audiences. Despite such software tending to cost money, it is developed to be as accessible as possible. Resources are partitioned to make intuitive user software. Without this key aspect, most programs would be unusable to non-programmers. Unfortunately, this is exactly where OSS cannot keep up:

> A very common problem among software developers is the fallacy that intuitiveness is a problem–specific rather than audience–specific… they don't bother simplifying anything that they regard as intuitive. This problem is particularly potent in Open Source projects due to members of the community investing time only in building what features they themselves would want to use and the fact that audiences are often not as well defined as they are in commercial software. (Levesque)

This is exactly where proprietary software sets itself apart from the OSS philosophy. Commercial software is built to be suitable for its users rather than the developer who made it. Although it is great to have a free alternative, free simply does not make the cut for being usable

by everyday users. In addition, commercial programs have, put simply, the time and resources to maintain their software appropriately. They guarantee long term support with their products, ensuring that bought software does not succumb to time. For example, “Sometimes open source software stops being maintained. Nobody's touched it in four years, and a bug is discovered. Who's going to fix it? How's that fix going to get merged? There are very few guarantees” (Ekberg). The reliability of OSS does not extend itself to regular users. Non-programmers do not have the ability to update, review and fix code. So when OSS projects run out of date, unlike proprietary software, people run out of options. These issues are not unaddressable however.

OSS still is a viable solution to prevent commercial software’s pitfalls, despite the current state of the OSS movement. Nor is action limited only to those in software development; everyday communities all across the world can make a difference. As a matter of fact the, “The best way to contribute to open source projects is to use the products themselves” (“How Non Programmers can Contribute to Open Source Code”). Although previously mentioned that OSS products are fairly hostile to non-programmers, utilizing the programs and providing your feedback and experiences sets the OSS movement back en route. OSS thrives on unique backgrounds, sharing experiences, serving communities, and most importantly, being used. The primary reason why people are not utilizing the benefits of OSS is because it is hostile to them; It is not as concerned with that as it is with having a free alternative to proprietary programs. This absence of usage only compounds itself. With less users, the factors that OSS postulates itself upon vanish as well. That is why, “Open Source technology continues to remain foreign to the large majority of computer users. In order to accommodate these users… the [issues of open source] must be seriously addressed and actively resolved” (Levesque). Such issues that hinder OSS’s success is what proprietary software accommodates for: It is user friendly, developed with

structure and feedback, and is brought to massive audiences' attention. By adopting proprietary software's methods of development, OSS has a shot at dominating the software industry. There is an urgency to address hostile user experience, knowledge based barriers and organization of open source. Projects fall flat due to lack of direction in development. Programs struggle to serve their users beyond the developers that made them. These are issues that proprietary software has evolved itself well beyond. Thus by evolving open-source to adapt to their users needs, to listen to their experiences, and to address audiences beyond themselves, OSS can compete with commercial software solutions. The burdens that proprietary software bears on their users can be resolved. Resolution can be achieved, especially alongside the communities that rely on software daily.

There are a multitude of ways that proprietary software burdens technology's applicability and progression. Encapsulating functionality, selling and exploiting user data, diminishing accessibility, stifling innovation, to name a few. All of which ultimately prevent the growth of software between communities, removing power from people's machines to developers. Yet in many forms, aspects are justifiable; commercial software uplifts accessibility through friendly user design, utilizing resources effectively to produce modern software for those who pay. Additionally, it empowers the technology industry by gaining the necessary funds to back important development. The fact that proprietary software has its benefits however, does not forgive its problems. So strictly speaking, deducing a definitive measurement to for-profit software's consequences is difficult. Regardless, its burdens are becoming increasingly obvious and cumbersome as time passes; it has undoubtedly compromised technology's growth and potential. To counteract its issues, OSS can be a powerful weapon. Encouraging innovation, accessibility, transparency and knowledge happens through simple usage. All can participate in

the development cycle of OSS. In order to ensure a brighter future for technology, people must recognize the flaws of proprietary software and pitch in; even if it means simply utilizing open-source software.

Works Cited

Ekberg, Chris. Personal Interview. 16 February 2026.

Garcia, Antero. "Technology might be making education worse." *Stanford Report*, 13 April 2023, https://news.stanford.edu/stories/2023/04/technology-might-be-making-education-worse. Accessed 25 February 2026.

Giddings, Scott. Personal Interview. 23 February 2026.

"How non programmers can contribute to open source code." *Open Source*, 24 October 2013, https://opensource.com/life/13/10/contribute-open-source-project-no-code. Accessed 25 February 2026.

Levesque, Michelle. "View of Fundamental Issues with Open Source Software Development." *First Monday*, 1 March 2004, https://firstmonday.org/ojs/index.php/fm/article/view/1137/1057. Accessed 25 February 2026.

Rudra, Sourav. "Android Security or Vendor Lock-In? Google's New Sideloading Rules Smell Fishy." It's Foss, 27 Aug 2025, https://itsfoss.com/news/new-android-sideloading-rules/. Accessed 20 April 2026.

Stallman, Richard. "Free Software Is Even More Important Now." *GNU Operating System*, 2025, https://www.gnu.org/philosophy/free-software-even-more-important.html. Accessed 25 February 2026.

"Technology and Society." *National Assessment Governing Board*, https://www.nagb.gov/naep-subject-areas/technology-and-engineering-literacy/framewor

k-archive/2014-technology-framework/toc/ch_2/society.html. Accessed 25 February 2026.